

02. Async / Await

Ek line me

Promises ke saath kaam karne ka aasan aur readable tarika.

`async` keyword

Function ke aage `async` lagao to wo **hamesha promise return karta hai**. Normal value ho to automatically promise me wrap ho jaati hai.

```
js

async function f() { return 1; }
f().then(console.log); // 1
```

`await` keyword

- Sirf `async` function ke andar chalta hai.
- Promise settle hone tak function ko **pause** karta hai, phir result deta hai.
- Pause hone se CPU block nahi hota, engine baaki kaam karta rehta hai.

```
js

async function f() {
  let result = await new Promise(res => setTimeout(() => res("done!"), 1000));
  console.log(result); // 1 sec baad "done!"
}
```

Kuch important points

- Normal (non-async) function me `await` likhoge to **SyntaxError**.
- Modules me **top-level await** chalta hai. Bina module ke ye trick use karo:

```
js

(async () => { let r = await fetch(url); })();
```

- `await` **thenable** objects (jinke paas `.then` method ho) bhi accept karta hai.
- Class method bhi `async` ho sakta hai: `async wait() { ... }`

Error Handling

- Reject hua promise `await` par **error throw** karta hai.
 - To normal `try...catch` use karo:
-

js

```
async function f() {  
  try {  
    let res = await fetch('http://no-such-url');  
  } catch (err) {  
    console.log(err);  
  }  
}
```

- `try..catch` nahi lagaya to async function ka returned promise reject ho jaata hai, uspe `.catch` lagao.

Promise.all ke saath

Kai kaam ek saath (parallel) karne ho:

js

```
let results = await Promise.all([fetch(url1), fetch(url2)]);
```

Ek reject hua to `Promise.all` turant reject ho jaata hai, **par baaki promises cancel nahi hote**. Unke errors uncaught reh sakte hain. Sab settle hone tak wait karna ho to `Promise.allSettled` use karo.

Yaad rakho

Keyword	Kaam
<code>async</code>	function ko promise return karwata hai + <code>await</code> allow karta hai
<code>await</code>	promise ka wait karta hai, error ho to throw karta hai

Top-level (async ke bahar) me `.then/.catch` hi use karne padte hain.